

Lecture 2: Agile, Scrum & DevOps

DM571 DM889: Software Engineering

Francesco Daghero

fdag@mmmi.sdu.dk

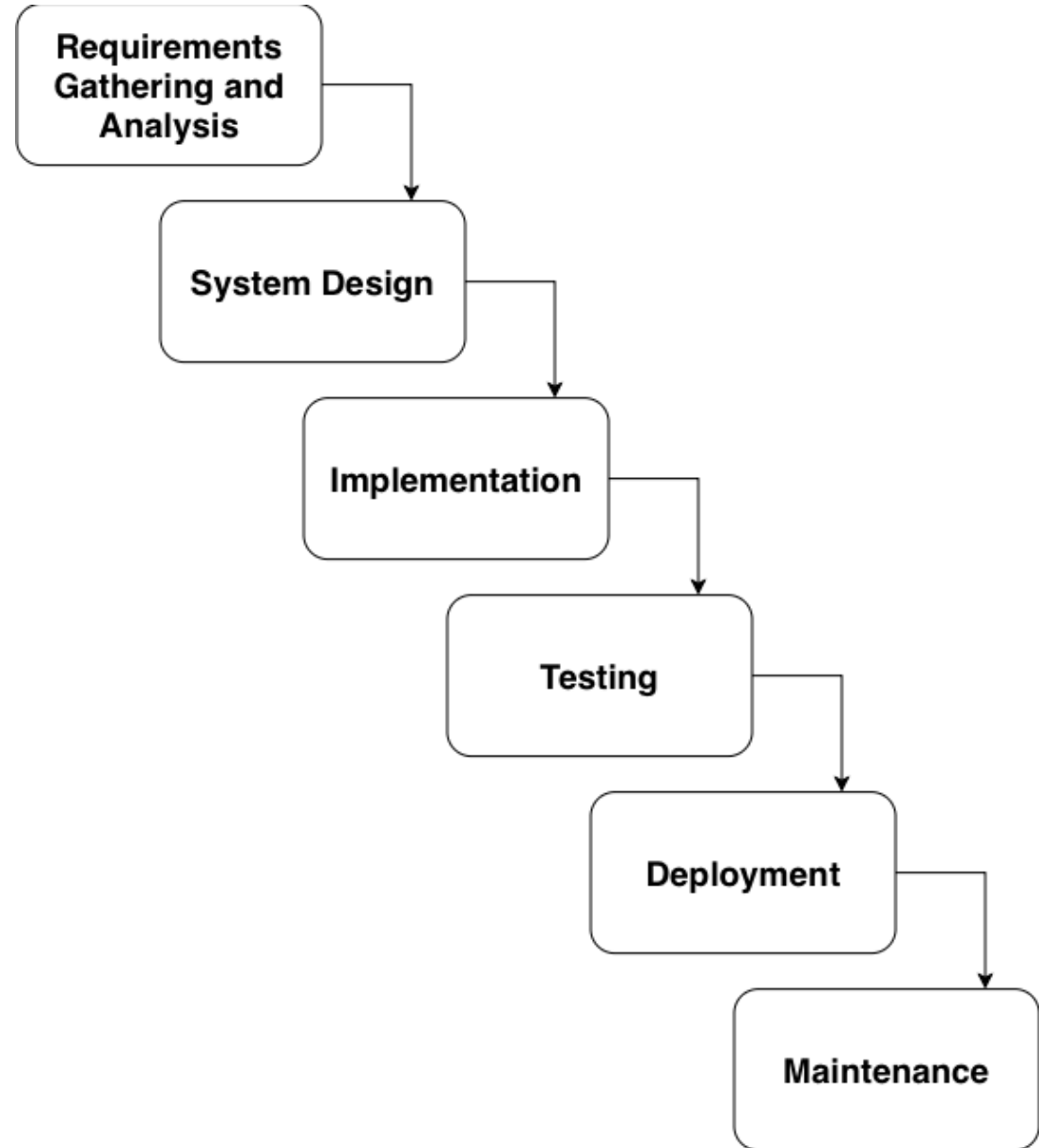
Tiziano Santilli

tisa@mmmi.sdu.dk

Recap

Waterfall Model

- Often credited to Dr. Royce
- Splits development process into phases
- Specialists in each phase
- "Big bang"
- "Obvious approach"
- Sequential



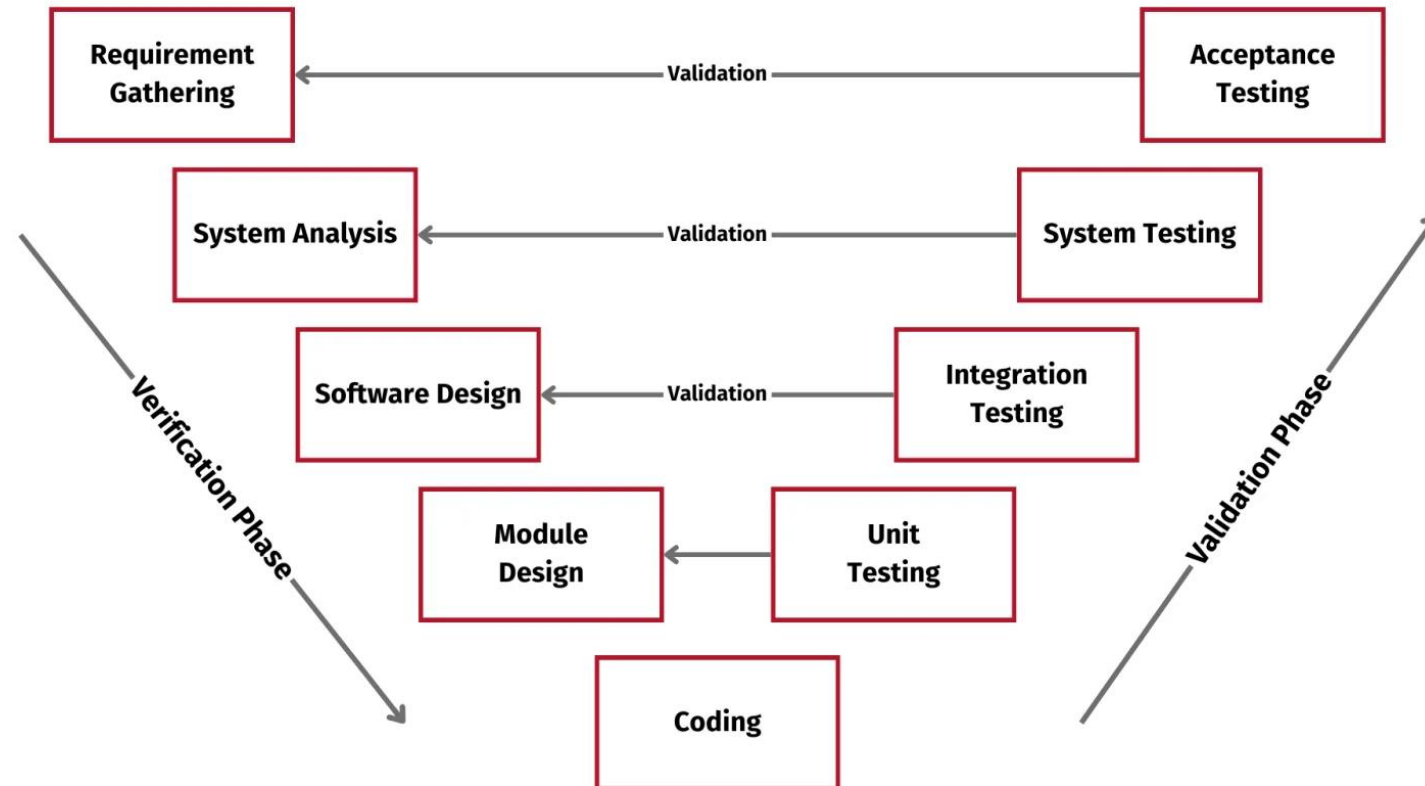
V-Model

Pros:

- Early Tests
- Direct Traceability
- Clear Governance

Cons:

- High Inflexibility
- Late Working Software
- Poor Fit for Ambiguity



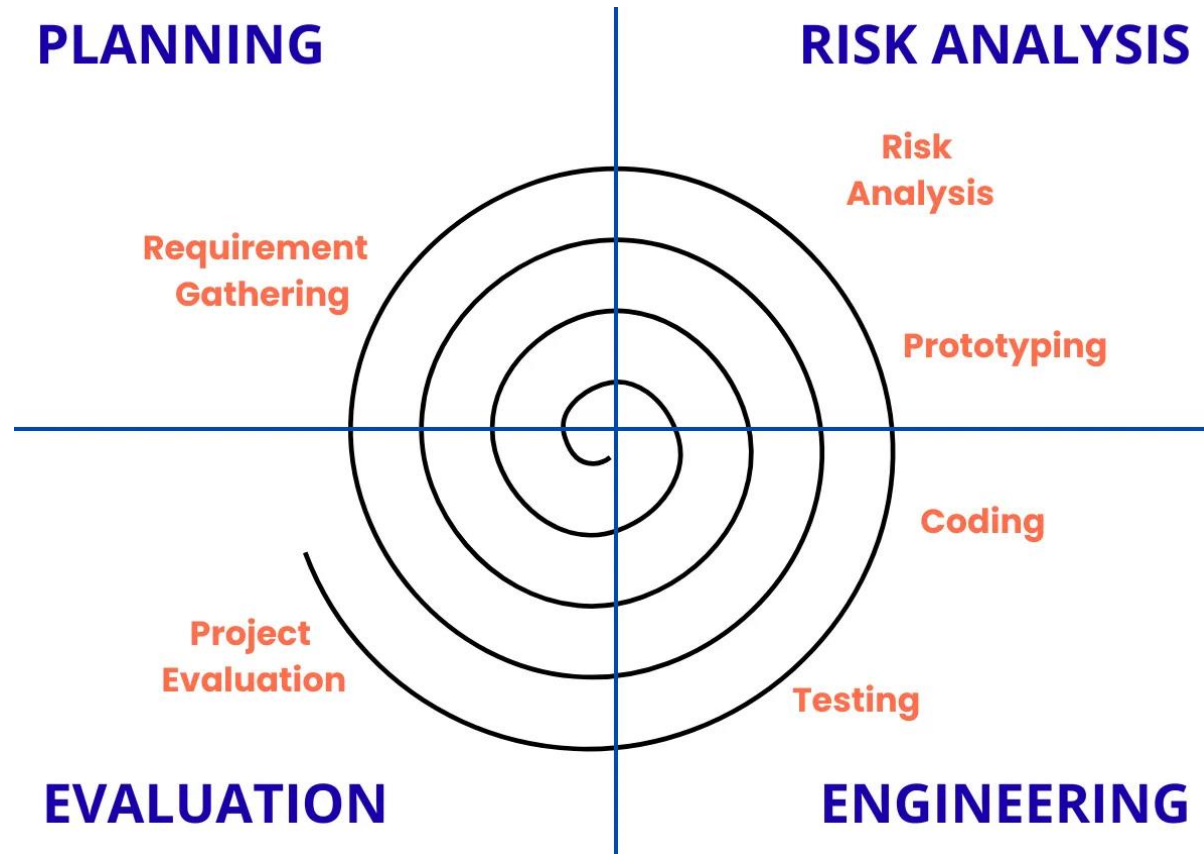
Spiral Model

Pros:

- Proactive Risk Mitigation
- Incremental Feedback
- High Adaptability

Cons:

- High Overhead, Cost
- Heavy Expertise Dependency
- Oversized for Standard Projects



Agile

Welcome to Agile or: Why Your Project Went Wrong)

Today's promise: by the end you will be able to explain Agile, run a sprint and write a decent user story

- Software projects have a historically miserable track record: large IT projects routinely run over budget, over time, and deliver less than promised.
- Classic failure pattern:
 - Month 1–6: write a 300-page requirements document
 - Month 7–24: build exactly that document
 - Month 25: deliver it
 - Month 25 + 1 day: "Oh... the business changed 18 months ago."
- **The core problem:** software requirements are discovered, not *collected*. Users often can't tell you what they want until they see something wrong.
- **Famous industry observation:** a large share of features in delivered software are rarely or never used.

Why Big Design Up Front Breaks

- **The cost-of-change curve (traditional view):** a defect or change costs orders of magnitude more the later it's found. Fix it in requirements = cheap. Fix it in production = expensive.
- Traditional response: *think harder, earlier*. Specify everything before writing code.
- Agile response: **flatten the curve instead**. Automated tests, continuous integration, modular design and short cycles make late change *cheap*, so you no longer need perfect foresight.
- Also fatal to waterfall:
 - **No working software until the end** → no feedback, no risk reduction, no learning.
 - **Integration hell** → everything meets for the first time in month 20.
 - **The 90% syndrome** → a project is "90% done" for 80% of its duration.

Snowbird, Utah, February 2001

- 17 people, a ski lodge, and a shared frustration with heavyweight, document-driven processes.
- They came from competing lightweight methods: Feature-Driven Development, Adaptive Software Development, Pragmatic Programming.
- They failed to agree on a single method. They *did* agree on a set of values.
- The result: **the Manifesto for Agile Software Development**, 68 words that changed the industry.

Fun fact: they also went skiing. The Manifesto is arguably the most influential document ever produced on a ski holiday.

The Four Values

Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

The Four Values, Decoded

- **Individuals & interactions > processes and tools:** A great team with a bad tool beats a bad team with Jira. Buying a tool is not adopting a process.
- **Working software > comprehensive documentation:** Documentation is not banned. *Unread* documentation is. Ask: "who will read this, and when?" If the answer is "nobody," don't write it.
- **Customer collaboration > contract negotiation:** Stop treating the client as an adversary you win points against. A fixed-scope contract punishes both sides when learning happens.
- **Responding to change > following a plan:** You still plan. You just don't confuse the plan with reality. *"Plans are worthless, but planning is everything."* Eisenhower

The 12 Principles (1–6)

1. **Satisfy the customer** through early and continuous delivery of valuable software.
2. **Welcome changing requirements**, even late in development, harness change for the customer's competitive advantage.
3. **Deliver working software frequently** weeks rather than months; prefer the shorter timescale.
4. **Business people and developers must work together daily** throughout the project.
5. **Build projects around motivated individuals** give them the environment and support they need, and trust them to get the job done.
6. **Face-to-face conversation** is the most efficient and effective method of conveying information within a development team.

The 12 Principles (7-12)

7. **Working software is the primary measure of progress.** Not lines of code. Not hours logged. Not slide decks.
8. **Sustainable development** sponsors, developers and users should be able to maintain a constant pace indefinitely. *Crunch is an anti-pattern, not a badge of honour.*
9. **Continuous attention to technical excellence and good design enhances agility.** Bad code makes you slow, and slow makes you unable to respond to change.
10. **Simplicity — the art of maximising the work not done — is essential.**
11. **The best architectures, requirements, and designs emerge from self-organising teams.**
12. **At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behaviour accordingly.**

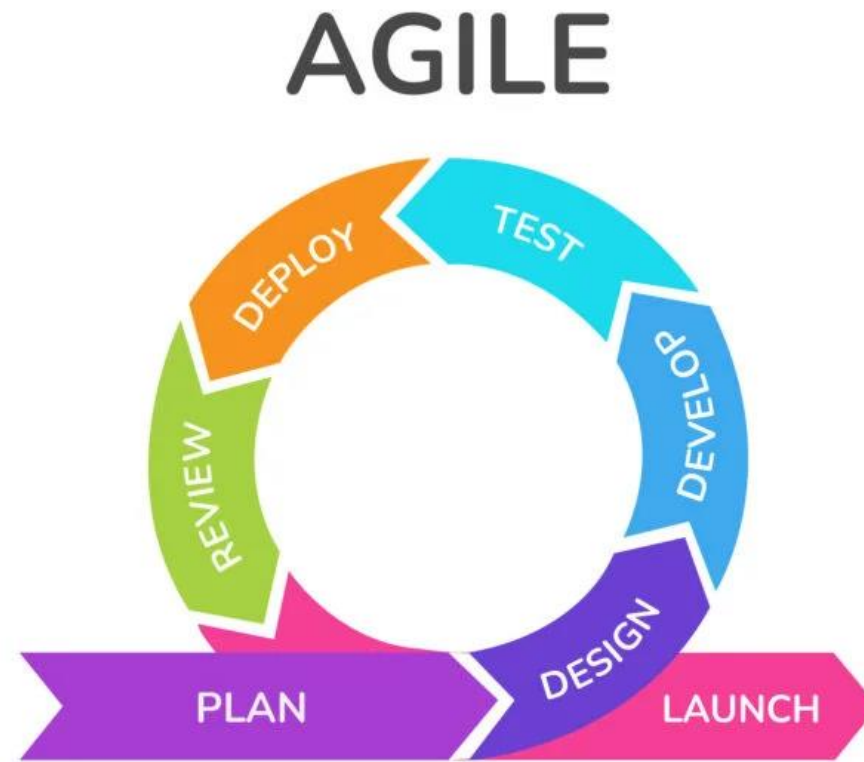
KISS



The Theory Underneath: Empirical Process Control

- **Defined process control:** you fully understand the process, so you can plan it precisely up front. Works for manufacturing 10,000 identical bolts.
- **Empirical process control:** the process is too complex/novel to predict, so you proceed by experiment. Works for software, where you never build the same thing twice.
- Empirical control rests on three pillars:
 - **Transparency** → everyone sees the same real state of the work.
 - **Inspection** → you look at actual output frequently.
 - **Adaptation** → you change what you're doing based on what you saw.
- **The Agile feedback loop:** Build → Measure → Learn → repeat. Short loops = fast learning = less risk.

The Agile Loop

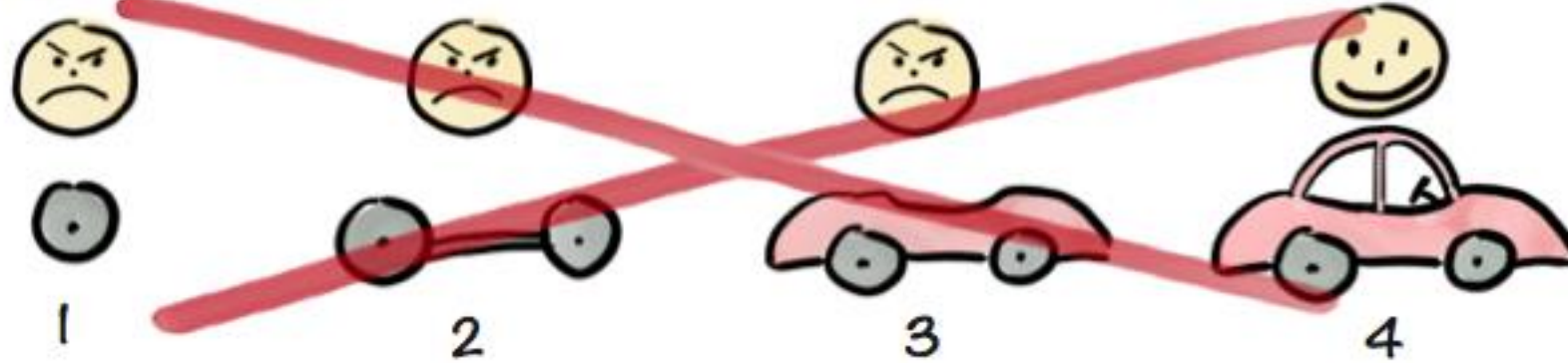


Iterative vs Incremental (Not the Same Thing!)

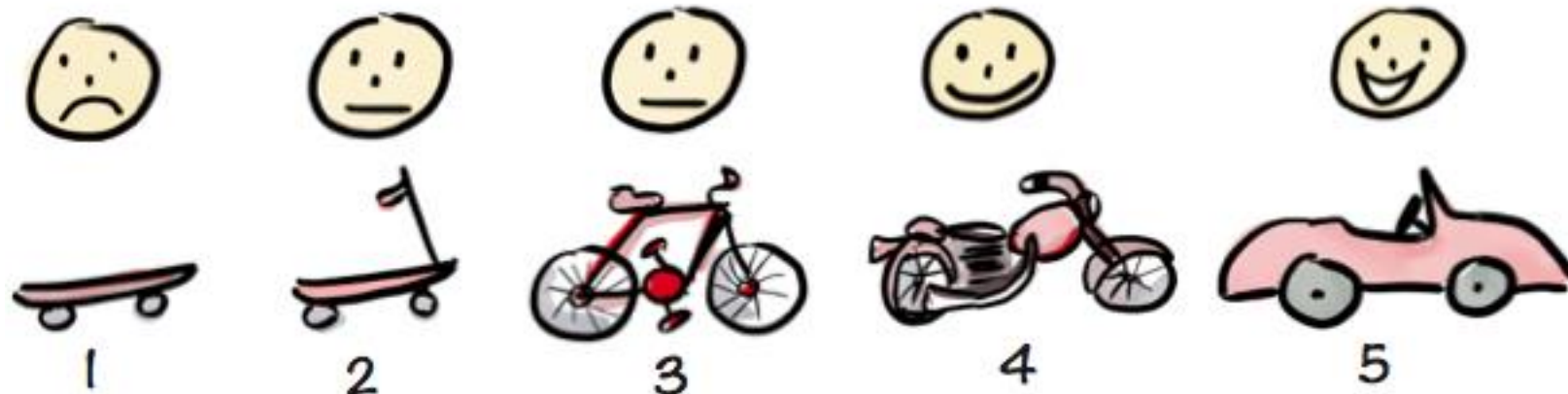
- **Incremental** = build it piece by piece. Finish the left third of the painting, then the middle, then the right.
- **Iterative** = build a rough whole, then refine it. Sketch the whole painting, then add shading, then detail.
- **Agile uses both.** Each cycle delivers a *thin slice of the whole system* that gets *refined over time*.
- Key term: **MVP (Minimum Viable Product)** the smallest thing you can build that produces real learning from real users. Emphasis on *viable*: a skateboard is viable, half a car is not.

Iterative vs Incremental: MVP Minimum Viable Product

Not like this....



Like this!



Exercise 1a: The Paper Airplane Factory Waterfall

Setup: groups of 4–5. Each group is a "factory."

Goal/Requirement: produce a paper airplane that **fly the most** (quality gate at 5 meters).

Waterfall Schedule: 10 minutes System Design, 5 minutes Implementation, 5 minutes Testing

Launch: 5 minutes

Exercise 1b: The Paper Airplane Factory Agile

Setup: groups of 4–5. Each group is a "factory."

Goal/Requirement: produce a paper airplane that **fly the most** (quality gate at 5 meters).

Agile Schedule: 3 Agile Loops, 5 minutes each (Design, Develop, Test, Review)

Launch: 5 minutes

Agile-Style Requirements : User Stories (1/2)

A user story is **a placeholder for a conversation**, not a specification.

The canonical template:

As a <type of user>, **I want** <some goal> **so that** <some reason/value>.

Examples:

- *As a returning customer, I want to save my payment details so that I can check out in one click.*
- *As a user, I want a MySQL table with an index on user_id.* ← That's a task, not a story. No user, no value.
- *As a user, I want the system to be good.* ← Meaningless. Untestable.

Agile-Style Requirements : User Stories (2/2)

The 3 C's (Ron Jeffries):

- **Card:** short enough to fit on an index card. Deliberately small.
- **Conversation:** the real requirement lives in the discussion with the customer.
- **Confirmation:** acceptance criteria that prove it's done.

Why the "so that" clause matters: it's the only part that tells you the *value*, which is how you prioritise, and sometimes reveals a much cheaper solution to the same need.

Is This a Good Story?

Use INVEST

Letter	Criterion	Test question
I	Independent	Can we build it without waiting for another story?
N	Negotiable	Is it a conversation starter, not a locked-down contract?
V	Valuable	Would a user or customer notice and care?
E	Estimable	Does the team know enough to size it? If not → spike/research task.
S	Small	Can it be finished comfortably within one iteration?
T	Testable	Can we objectively decide "done" or "not done"?

Acceptance Criteria: Given / When / Then

Acceptance criteria turn a fuzzy story into something **testable**.

Story: *As a registered user, I want to reset my forgotten password so that I can regain access to my account.*

Scenario: Successful reset request

GIVEN I am on the login page

AND I have a registered account

WHEN I click "Forgot password" and submit my email

THEN a reset link is sent to that email

AND the link expires after 60 minutes

Scenario: Unknown email address

GIVEN I am on the "Forgot password"

page WHEN I submit an email that is

not registered

THEN I see the same confirmation

message

AND no email is sent

Sprint: Definition of Ready vs Definition of Done

A **sprint** is a short, fixed-length time period (usually one to four weeks) when an Agile team works to complete a specific set of tasks

Definition of Ready a story may enter a sprint and become a task only if:

- Written in story format with clear value
- Acceptance criteria agreed
- Dependencies identified
- Estimated by the team
- Small enough to fit in one sprint

Definition of Done (DoD) a shared, team-wide checklist.

A typical one:

- Code written and peer-reviewed
- Unit tests written and passing; coverage threshold met
- Integration tests passing
- No new critical static-analysis warnings
- Documentation / API docs updated
- Deployed to staging and accepted by the Product Owner
- No known defects of severity

Scrum

Scrum is the most widely used Agile framework. **Three roles, three artifacts, five events.**

Scrum: The Roles

Role	Owns	Common failure
Product Owner	What and why. The Product Backlog, its ordering, and maximising value. One person, not a committee.	A PO who is unavailable, or a proxy with no authority to decide.
Developers	How. Self-managing; collectively own the Sprint Backlog and the Increment; create the Definition of Done.	Being told how to work; secret specialist silos.
Scrum Master	The process. A servant-leader who coaches the team, facilitates events, and removes impediments.	Becoming a project manager, task-assigner, or meeting-scheduler.

Critically: the Scrum Master is **not the boss**, and the Product Owner is **not the boss**.

The team is self-managing
it decides *how much* to take on and *how* to build it.

Nobody outside the team
may change the Sprint Backlog mid-sprint.

Scrum: The Artifacts

Artifact	What it is	Commitment
Product Backlog	Ordered, living list of everything that might be needed. Never "complete."	Product Goal
Sprint Backlog	The items selected for this sprint + the plan for delivering them.	Sprint Goal
Increment	The usable, potentially releasable sum of all work done, meeting the DoD.	Definition of Done

Backlog refinement (grooming) an ongoing activity, roughly 5–10% of team capacity: splitting large items, adding detail, re-estimating, re-ordering. Keeps the top of the backlog always "Ready."

The Sprint Goal is the underrated hero. It's a single sentence of why this sprint matters (e.g., "Enable students to complete a rental end-to-end on mobile"). It gives the team room to negotiate scope while still succeeding. If you drop one story but achieve the goal, the sprint succeeded.

Scrum: The Five Events

Event	Timebox (1-month sprint)	Purpose
The Sprint	≤ 1 month (commonly 2 weeks)	The container for all other events. Fixed length. Never extended.
Sprint Planning	≤ 8 h	Why (Sprint Goal), what (select items), how (plan the work).
Daily Scrum	15 min, same time & place	Developers re-plan the next 24 h toward the Sprint Goal.
Sprint Review	≤ 4 h	Demo the Increment to stakeholders; inspect the product; adapt the backlog. A working session, not a presentation.
Sprint Retrospective	≤ 3 h	Inspect the process; agree concrete improvements.

Agile Limits

When Agile is a poor fit:

- Safety-critical / heavily regulated systems requiring up-front certification (though hybrids exist and are growing).
- Genuinely fixed-scope, fixed-price contracts with no willing customer collaboration.
- Trivial, fully understood, one-off work. The overhead isn't worth it.
- Organizations unwilling to let teams self-manage. **Agile inside a command-and-control culture becomes theatre.**

Agile is a response to uncertainty, not a productivity trick.

DevOps & DevSecOps

Agile Got Us Halfway

- Agile taught us to build the right thing, in small pieces, with fast feedback.
- **Uncomfortable follow-up:** so what? A finished feature sitting on a developer's laptop has delivered exactly zero value. Principle 1 of the Manifesto says deliver valuable software, not write it.
- Agile optimized **Dev**. Then the work hit a wall.

Development	Operations
Paid to deliver change	Paid to deliver stability
Success = velocity	Success = uptime
"Ship the feature!"	"Nothing break the system!"

Who has written code that worked locally and broke instantly on someone else's machine?

DevOps

Definition: DevOps is a set of cultural practices, workflows and tools that shortens the time from commit to safely running in production, and shortens the feedback loop from production back to the developer, by making development and operations a single team with shared ownership.

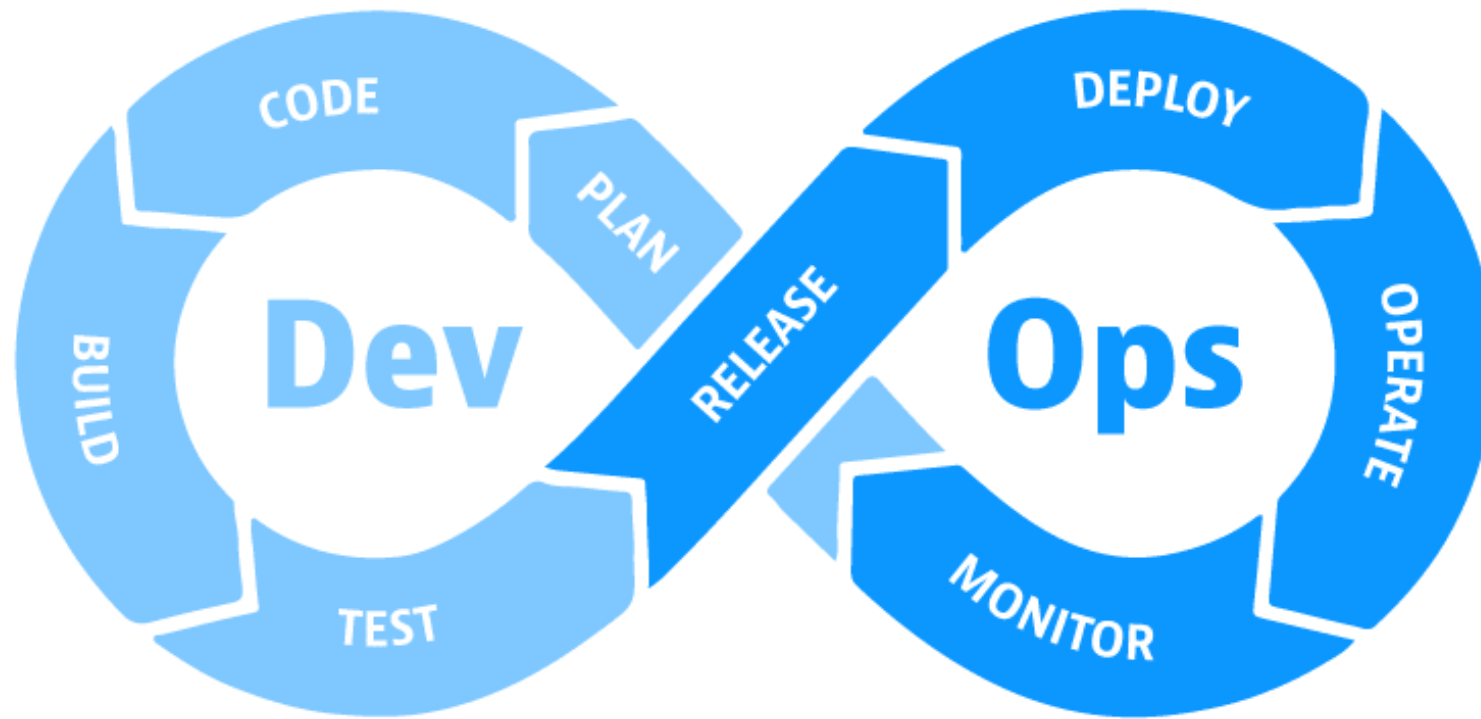
The one-line version: *"You build it, you run it."*

What DevOps is NOT — all four of these are common and all four are wrong:

- A job title. "We hired a DevOps." No, you hired a sysadmin with Kubernetes on their CV.
- A separate DevOps team between Dev and Ops
- A tool you buy.
- "Developers do ops now, so we can fire the ops team."

The relationship to Agile: DevOps is not Agile's successor, it is the same idea (small batches, fast feedback, shared responsibility) applied to everything after the code is written.

DevOps



DevSecOps

Definition: DevSecOps extends DevOps by making security a shared responsibility of everyone, automated into the pipeline, and starting at design time rather than a manual audit performed by a separate team at the end.

Why DevOps structurally fails:

- Security is the last gate before a deadline, guaranteed to be overruled.
- An architectural flaw found at the end is enormously expensive; found at design time it is a conversation.
- One security engineer per 100+ developers cannot review everything.

DevSecOps

